

squatch Plan Review Prompts

Build-It Review	2
Compression Review	4
Cut Review	7
Kill Review	10

Build-It Review

Source: BuildItReview.md

What this tests and why

A design document that claims to be self-contained -- an implementer needs no other document -- can pass any number of adversarial READINGS and still fail the first WRITING: reviewers evaluate arguments, implementers consume decisions, and only the second exposes what the text does not determine. This exercise tests the claim under writing. Fresh agents with no project context role-play the implementer for **<PHASE_SCOPE>** and must produce skeletons from the document at **<PLAN_PATH>** alone. Every decision the document forces them to invent is a measured self-containment gap. Convergence is the signal: a decision independently invented by two or more isolated agents is a real spec gap, not one agent's taste. Designed to re-run as the document evolves -- rescope **<PHASE_SCOPE>** to the earliest unbuilt phases each time -- and a near-empty ledger is a valid outcome, not a failed run.

Run shape

- 3 DRY-RUN agents, identical prompt (below), fully isolated: each gets the document at **<PLAN_PATH>** and nothing else -- no repo browsing, no conversation history, no prior review results, no knowledge that other agents exist. Identical prompts keep the convergence statistic clean; withholding prior rounds prevents anchoring on old gaps.
- 1 SYNTHESIS agent (prompt below) compares the three ledgers after all finish.
- High reasoning effort for all four. No file writes by any agent; output returns as structured data.

Dry-run agent prompt (verbatim)

You are the implementing engineer for the system specified by the design document at **<PLAN_PATH>**. Your ONLY input is that document. Read all of it. You cannot ask questions; there is no other documentation, no repository, and no team.

Produce three things for **<PHASE_SCOPE>** as defined in the document's implementation-phases section:

1. REPO SKELETON: the module/package layout you would create, one line of responsibility per module. Cover every deliverable the document names for **<PHASE_SCOPE>**.
2. INTERFACE STUBS: signatures in the document's implementation language (classes, functions, data shapes, event shapes, config-schema sketch) for the document's core contracts and the **<PHASE_SCOPE>** modules, faithful to the document. Stubs are evidence of your reading, not production code -- no bodies beyond a placeholder.
3. INVENTED-DECISIONS LEDGER -- the actual product of this exercise. Every place you had to make a choice the document did not determine for you, one entry each:
 - decision: what you had to decide, in one sentence.
 - section: the section (by number/heading) that should have determined it.
 - chose: what you picked, in one sentence, and why.
 - class: one of

- missing (the document is silent),
- ambiguous (two readings are defensible),
- contradictory (two passages disagree),
- deliberate-freedom (the document plausibly leaves this to the implementer on purpose -- naming these calibrates the other three).

Also produce a BLOCKED list: anything you could not even invent your way past -- where any choice you made would risk violating a stated contract.

Rules: implement AS SPECIFIED -- do not evaluate, improve, or redesign the system; if you disagree with a decision the document makes, follow the document. Filling a gap from your own experience with similar systems is still an invented decision -- ledger it; background knowledge masking a gap is exactly the failure this exercise measures. Do not pad the ledger: a genuinely determined decision does not belong in it, and an empty BLOCKED list is a valid answer. Be exhaustive on the ledger before polishing the stubs; if effort runs short, cut stub detail, never ledger entries.

Synthesis agent prompt (verbatim)

You receive three invented-decisions ledgers plus blocked lists, produced independently by three isolated implementers from the same design document at `<PLAN_PATH>`. Read the document, then:

1. Cluster ledger entries describing the same underlying decision across ledgers (they will word it differently).
2. Rank clusters: (a) any BLOCKED entry first; (b) then decisions invented by 3/3 agents, then 2/3; (c) within a tier, contradictory > ambiguous > missing. Entries by a single agent classed deliberate-freedom are noise -- drop them; single-agent entries otherwise go in a low-confidence tail.
3. For each surviving cluster: name the decision, the document section that should own it, what the three implementers each chose (divergence matters -- three different inventions is worse than three matching ones), and a one-sentence proposed document edit that would have determined the decision.
4. Verify each cluster against the document before reporting it: if the document actually does determine the decision and the implementers missed it, report it separately as a FINDABILITY problem (the answer exists but implementers did not find it -- a structure defect, not a content gap).

Output, verdict first:

- Self-containment: SELF-CONTAINED (no verified BLOCKED entries, no multi-agent clusters beyond deliberate-freedom) / GAPS-FIXABLE (multi-agent clusters exist and every one has a determinable proposed edit) / NOT SELF-CONTAINED (any verified BLOCKED entry, or any contradictory cluster confirmed against the text) -- with one paragraph of justification.
- The ranked gap list with per-cluster convergence counts, the findability list, and the low-confidence tail.

Disposition of results

Gaps and findability problems become document edits, reviewed before applying, same as every round; the exercise re-runs after material edits, rescoped to the phases now earliest-unbuilt. Skeletons and stubs are discarded -- they are measurement instruments, not the start of the implementation. Nothing from this exercise commits code.

Compression Review

Source: CompressionReview.md

What this tests and why

A design document accretes prose: rationale and rhetoric earn their keep during review rounds, but the document's standing audience is the implementer, and prose taxes them -- every sentence of motivation is a sentence they must read to confirm it contains no requirement. This exercise compresses the document at `<PLAN_PATH>` to specification form: every technical fact survives at full strength, everything addressed to a reviewer rather than an implementer goes. It is designed to be re-run whenever prose has accreted; on an already-compressed document the correct outcome is little or no change, and that is a valid result, not a failed run. Compression is only trustworthy if loss is measured, so an independent agent audits the result against the original. The compressor changes how things are said, never what is specified.

Run shape

- 1 COMPRESSOR agent (prompt below): gets the document at `<PLAN_PATH>`, writes the draft spec to a scratch path (`<SPEC_PATH>`), returns its ledger and metrics as structured data. No other file writes.
- 1 FIDELITY agent (prompt below): runs after the compressor finishes; gets the original, the draft, and the compressor's judgment-call ledger. Independent -- it did not see the compressor's reasoning, only its output.
- High reasoning effort for both. The fidelity agent may use read-only tools (diff, grep) to check verbatim blocks mechanically.

Compressor prompt (verbatim)

You are rewriting the design document at `<PLAN_PATH>` into a specification, written to `<SPEC_PATH>`. The document's content is settled; your job is form only: remove rhetoric and narrative prose, retain every technical fact at full strength. You change how things are said, never what is specified. The document may have been compressed before -- do not force changes where none are needed; passages already in spec form pass through untouched.

Classify every sentence into one of three classes:

1. **NORMATIVE** -- contracts, invariants, interfaces, event shapes, schemas, names, defaults, limits, thresholds, orderings, deliverables, commands, refusals ("not building X"), anything an implementer could violate or get wrong. Keep, verbatim or tightened. Tightening may not weaken, strengthen, generalize, narrow, or reorder a requirement.
2. **RATIONALE** -- why a decision was made, alternatives considered and rejected, failure history. Compress each to a one-line "Why:" note attached to the decision it justifies. Rationale that names a rejected alternative or a past failure survives -- it prevents re-litigating the decision. Rationale that only motivates or reassures is rhetoric.
3. **RHETORIC** -- persuasion, motivation, analogy, hedging, restatement that adds nothing to a statement already made (normative content repeated across sections falls under the deduplication rule below, not here), throat-clearing, anything whose audience is a reviewer rather than an implementer. Delete.

Form rules:

- Spec voice: declarative, present tense, no first person, no rhetorical questions. Use MUST / MUST NOT / SHOULD / MAY exactly where the original expresses that level of obligation -- do not upgrade a "should" to a MUST or downgrade a "must" to a SHOULD.
- Prefer tables and lists over paragraphs wherever content is enumerable.
- Section numbers and heading text stay exactly as in the original -- other documents and tooling may cite sections by number or heading, and every internal cross-reference must still resolve.
- VERBATIM-PROTECTED, byte-for-byte: all fenced code blocks, commands, file paths, config examples, schemas, and any region delimited by extraction markers (paired BEGIN_X / END_X lines or equivalent), including the markers themselves -- tooling may extract these regions literally and any edit breaks it.
- Deduplication: when the same normative content appears in more than one section, keep one canonical statement in the section that owns the topic and replace every other occurrence with a cross-reference to it. Merge only duplicates that are identical in substance: if two occurrences differ in any substantive way -- obligation force, scope, a name, a default, a condition -- they are not duplicates but a latent contradiction; keep both verbatim, do not reconcile them, and record the pair under DIVERGENT DUPLICATES. Choosing a winner is design, not compression.
- No new content: do not add requirements, defaults, clarifications, or examples the original does not state. If a passage is ambiguous, carry the ambiguity into the spec unresolved and flag it in the ledger -- compression must not smuggle in design.

Also produce, as structured output (not written into the spec):

- JUDGMENT-CALL LEDGER: every sentence that resisted classification -- quote, section, the class you chose, why. Include anything you kept despite it reading as rhetoric because it carried a fact, and anything you deleted despite it brushing against a requirement.
- FLAGGED AMBIGUITIES: passages carried forward unresolved, per the no-new-content rule.
- DUPLICATE MAP: every merge performed -- the location of each original occurrence and the canonical spec statement that now serves them all.
- DIVERGENT DUPLICATES: occurrences that repeat the same content but differ in substance, kept unreconciled -- both locations, both quotes, and the difference.
- METRICS: per-section line counts before and after, and the overall ratio.

Rules: content is settled -- if you disagree with a decision, compress it faithfully. When keep-vs-delete is in doubt, keep and flag: a fat spec is recoverable from the ledger, a silently dropped contract is not. Be exhaustive on classification before polishing wording; if effort runs short, leave a section verbatim and say so -- never summarize one from memory.

Fidelity agent prompt (verbatim)

You receive an original design document at `<PLAN_PATH>`, a compressed specification at `<SPEC_PATH>`, and the compressor's judgment-call ledger and duplicate map. Assume the compression is lossy until the text proves otherwise. Work the ORIGINAL side, section by section: for every normative statement in the original, find its counterpart in the spec. Do not audit spec-side first -- that finds inventions but never drops.

Counterparts may be many-to-one: the duplicate map lists where the compressor merged repeated content into one canonical statement. Verify each canonical statement against EVERY original occurrence

it serves -- a nuance present in one occurrence and absent from the canonical statement is DROPPED; if the occurrences diverged and the spec commits to one reading, that is RESOLVED AMBIGUITY. A merge absent from the duplicate map is still a merge -- audit it the same way, and note the missing map entry.

Report every instance of:

1. DROPPED -- a normative statement in the original with no surviving counterpart.
2. ALTERED -- obligation force changed (must vs should), a name, default, limit, or ordering changed, a condition's scope narrowed or broadened.
3. INVENTED -- the spec states something the original does not determine.
4. RESOLVED AMBIGUITY -- the original supports two readings and the spec commits to one.
5. VERBATIM VIOLATION -- any protected region (fenced code, commands, schemas, extraction-marker blocks) that is not byte-identical; check these mechanically with diff, not by eye.
6. BROKEN REFERENCE -- a section number, heading, or internal cross-reference that no longer resolves.

Every finding: original location, spec location (or "absent"), both quotes, class. Compressor ledger entries do not excuse a finding -- a flagged loss is still a loss; note the flag.

Verdict: LOSSLESS / LOSSY-MINOR (findings are recoverable wording issues) / LOSSY (a contract, default, or invariant was dropped or altered), with the finding count per class. An empty finding list is a valid answer only after a complete original-side pass -- say which sections you covered.

Disposition of results

Findings become edits to the draft spec, reviewed before applying, same as every prior round; the fidelity pass re-runs after edits. DIVERGENT DUPLICATES are content defects, not compression defects: they route to the document's normal review process for the owner to reconcile, and stay unreconciled in the spec until then. Nothing replaces the original automatically: the draft is promoted only on a LOSSLESS verdict plus a human-reviewed diff, keeping the original's file name (other tooling may reference it) and with the prior version retained in git history. The judgment-call ledger and metrics are kept with the review record; the scratch draft is otherwise discarded.

Cut Review

Source: CutReview.md

What this tests and why

This review finds what the design document at `<PLAN_PATH>` specifies without justification. The stance is adversarial in the direction reviews usually are not: every feature is assumed unjustified until it traces to a goal the document states. Gold plating, premature generality, and misordered effort survive adversarial reading because they are defects of value, not correctness -- nothing is wrong, there is just more plan than the goal requires, or the right plan in the wrong order. One caveat is structural: a missing-item finding is falsified by the document, but a removal finding is falsified by the future -- no reviewer can prove a feature will never be needed. This review therefore produces candidates with steelmen, not verdicts of waste; the owner decides. Designed to re-run as the document evolves; on a lean document, few findings is a valid outcome, not a failed run.

Run shape

- 3 CUT agents, identical prompt (below), fully isolated: each gets the document at `<PLAN_PATH>` and nothing else -- no repo browsing, no conversation history, no prior review results, no knowledge that other agents exist. Identical prompts keep the convergence statistic clean; withholding prior rounds prevents anchoring on old findings.
- 1 SYNTHESIS agent (prompt below) merges the three reports after all finish.
- High reasoning effort for all four. No file writes by any agent; output returns as structured data.

Cut agent prompt (verbatim)

You are a skeptical senior engineer reviewing the design document at `<PLAN_PATH>` for scope, not correctness: your job is to find the features that should not be built -- gold plating, premature generality, low return on the time they cost -- and the effort spent in the wrong order. Every feature is unjustified until the text justifies it. Your only input is the document: no other documentation, no repository, no team, no questions.

First, extract the yardstick from the document itself:

- Its stated goals, anti-goals, and any explicit refusals list ("not building X") -- these, not your taste, define justified. The refusals list also calibrates where the owner's bar for exclusion sits.
- Its optimization metric: what the document says it is minimizing or maximizing (time to first working run, operator minutes per change, concept count, ...). If no metric is stated, report NO-METRIC as your lead finding -- do not invent one; without it, do the traceability tier below but skip the ranking tier, and say so.

Then two tiers of findings:

TRACEABILITY (needs no metric): for every feature and deliverable, establish the chain feature -> requirement -> stated goal. Report each feature that traces to nothing, classed as one of:

- gold-plating (exceeds what the stated goal requires),
- premature generality (built for futures the document does not commit to),

- redundant capability (overlaps another feature that already serves the goal),
- speculative robustness (handles failure modes the stated scope cannot produce).

RANKING (needs the metric): rank features ordinally against each other -- no absolute estimates, no hours -- on cost (from the text: dependency counts, deliverable lists, protocol surface) versus contribution to the stated metric, every rank citing the passages that drive it. Report:

- inversions: a high-cost, low-contribution feature sequenced ahead of a cheap, high-contribution one -- the implementation-phases section is the main target;
- simplifications: merge (two mechanisms where one serves), hardcode (configurability that could be a constant), flatten (an abstraction layer with a single implementation).

Every finding, both tiers:

- cites a line number or heading -- no finding without a location;
- carries a STEELMAN: search the WHOLE document for the load the feature bears -- rationale notes, failure history it guards against, sections that depend on it -- and quote the strongest defense you find. A cut that would break a stated contract or another section's dependency is not a cut; report it as COUPLED and move on. Risk rule: a feature whose benefit is preventing a failure the document names is evaluated against that failure's cost, not the metric -- prevention looks like gold plating until the incident;
- states the loss: what the plan gives up if the finding is applied, and why the stated goals survive that loss. A finding that cannot name its loss is padding -- drop it;
- proposes ONE disposition: cut (delete), defer (move to a named later phase), refuse (add to the refusals list -- stronger than cut, it prevents re-accretion), shrink (keep the requirement, drop this version of it), or resequence / merge / hardcode / flatten, with the specific edit.

Rules:

- Scope only -- correctness defects, gaps, and ambiguities are out of scope here; note them in one line and move on.
- Separate "the text does not justify this" (fact) from "I would not build this" (opinion) and label each.
- Do not pad: a justified feature is not a finding, and a lean document yields a short report.

Synthesis agent prompt (verbatim)

You receive three cut reports produced independently by isolated reviewers from the same design document at <PLAN_PATH>. Read the document, then:

1. Cluster findings describing the same underlying feature or sequencing decision (they will word it differently).
2. Rank clusters: any NO-METRIC finding first (a document defect that gates the rest); then 3/3 convergence, then 2/3; within a tier, untraceable > inversion > simplification. Single-reviewer findings labeled opinion drop; other single-reviewer findings go in a low-confidence tail.
3. Verify each cluster against the document by re-running the steelman search yourself: if the document does justify the feature and the reviewers missed it, move the cluster to the FINDABILITY list -- the justification exists but sits where a reader would not look. Where reviewers disagreed on a feature's rank or disposition, keep the cluster and report the disagreement -- divergent value judgments are a

finding for the owner, not noise to average away.

4. Assemble the MINIMAL PLAN: the subset and ordering of features that reaches the document's stated goals -- and its metric, if stated -- soonest, with everything outside it carrying its cluster's disposition.

Output, verdict first:

- Scope: LEAN (no multi-agent cluster survives verification) / TRIMMABLE (clusters survive and every disposition preserves the stated goals) / GOLD-PLATED (at least one 3/3 cluster of an untraceable feature whose steelman found no load), with NO-METRIC flagged separately if found, and one paragraph of justification.
- The ranked cluster list with convergence counts and dispositions, the minimal plan, the findability list, and the low-confidence tail.

Disposition of results

Findings become document edits, reviewed before applying, same as every round -- refuse-class edits go to the refusals list so cut scope does not re-accrete. NO-METRIC, if found, is a content gap for the owner: the review re-runs at full strength once the document states what it optimizes. The exercise re-runs after material edits, and cuts carry a natural check: a feature cut here that later review or implementation independently re-invents as a missing decision has defended itself -- it returns. The cut decision belongs to the owner, informed by steelmen and convergence counts -- the review recommends, it does not decide. Nothing from this exercise commits code.

Kill Review

Source: KillReview.md

What this tests and why

This review decides whether the design document at `<PLAN_PATH>` should be built as written. The stance is adversarial: the plan is assumed flawed until the text proves otherwise, and the reviewer's job is to find what sinks it, not to reassure. Independent replication separates real defects from one reviewer's taste: a problem found by two or more isolated reviewers is real, a single-reviewer finding is a lead. Designed to re-run as the document evolves; on a mature document, few findings is a valid outcome, not a failed review.

Run shape

- 3 KILL agents, identical prompt (below), fully isolated: each gets the document at `<PLAN_PATH>` and nothing else -- no repo browsing, no conversation history, no prior review results, no knowledge that other agents exist. Identical prompts keep the convergence statistic clean; withholding prior rounds prevents anchoring on old findings.
- 1 SYNTHESIS agent (prompt below) merges the three reports after all finish.
- High reasoning effort for all four. No file writes by any agent; output returns as structured data.

Kill agent prompt (verbatim)

You are a skeptical senior engineer whose job is to decide whether to KILL the plan at `<PLAN_PATH>` before anyone builds it. Assume it is flawed until the text proves otherwise. Deliver the problems, not reassurance. Your only input is the document: no other documentation, no repository, no team, no questions.

Lead with a verdict, no preamble:

- Viability: VIABLE / VIABLE-WITH-CHANGES / NOT VIABLE AS WRITTEN. NOT VIABLE requires at least one finding that no plausible edit short of a redesign would fix -- name it.
- Confidence: high/med/low, and what specific evidence would raise it.

Then findings, ranked by how likely each is to sink the project:

- Soundness failures: logic that does not hold, decisions that contradict each other, steps that cannot be built in the stated order.
- Gaps: what MUST be specified to execute and is not (cite the section that should contain it).
- Unstated assumptions: what has to be true for this to work, which of those are unproven, and what happens if each proves false.
- Weakest link: the single element most likely to fail, and the blast radius if it does.

Every finding:

- cites a line number or heading -- no finding without a location;

- states a concrete failure scenario: what breaks, under what conditions, observed how. A finding that cannot name its failure is an opinion -- label it as one;
- is verified against the WHOLE document before you report it: search for the answer elsewhere first. If the document does answer it but the answer sits where a reader would not look, report it separately as FINDABILITY -- a structure defect, not a content gap;
- ends with a one-sentence edit that would resolve it, if one exists.

Rules:

- Evaluate the plan as written. Separate "this is wrong" (fact) from "I'd do it differently" (opinion) and label each; redesign proposals are out of scope.
- No praise unless it is load-bearing to the verdict. Skip strengths sections.
- Do not pad: if something is genuinely fine, say "no issue found"; if a section is too vague to evaluate, say "unassessable -- missing X" rather than guessing charitably.

Synthesis agent prompt (verbatim)

You receive three kill reports produced independently by isolated reviewers from the same design document at `<PLAN_PATH>`. Read the document, then:

1. Cluster findings describing the same underlying defect across reports (they will word it differently).
2. Rank clusters: any finding cited as grounds for NOT VIABLE first; then 3/3 convergence, then 2/3; within a tier, soundness > gap > assumption. Single-reviewer findings labeled opinion drop; other single-reviewer findings go in a low-confidence tail.
3. Verify each cluster against the document before reporting it: if the document actually resolves the issue and the reviewers missed it, move it to the FINDABILITY list.
4. Reconcile the verdicts: if they disagree, state whether the disagreement comes from different findings or from different weightings of the same findings -- the second kind is a judgment call to surface for the owner, not to resolve yourself.

Output: the reconciled verdict with any disagreement explained, the ranked defect list with per-cluster convergence counts, the findability list, and the low-confidence tail.

Disposition of results

Defects and findability problems become document edits, reviewed before applying, same as every round; the review re-runs after material edits. The kill decision itself belongs to the owner, informed by the reconciled verdict -- the review recommends, it does not decide. Nothing from this exercise commits code.